



LI.FI Security Review

IntentExecutor(v2.0.0)

Security Researchers

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

April 16, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Low Risk	4
6.1.1	Function <code>executeAndSweep()</code> silently strands ETH received during execution	4
6.2	Gas Optimization	4
6.2.1	Redundant approve calls on already-maxed allowances waste gas	4
6.3	Informational	5
6.3.1	Residual or accidentally sent tokens in <code>IntentExecutor</code> can be stolen by anyone	5

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Uniswap, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Aztec, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Buck Labs and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- solidity/src/libs/IntentExecutor.sol

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 8 hours in total, [LI.FI](#) engaged with Sujith as an independent researcher to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 3 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/catapultar
Commit	211d261
Audit Timeline	April 6, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Gas Optimizations	1
Informational	1
Total Issues	3

6 Findings

6.1 Low Risk

6.1.1 Function `executeAndSweep()` silently strands ETH received during execution

Context: [IntentExecutor.sol#L79](#)

Description: Function `executeAndSweep()` has no native ETH handling. If any call in the batch produces ETH output (WETH unwrap, DEX refund, protocol rebate, etc.), the ETH lands in the `IntentExecutor` via `receive()` and remains stranded indefinitely. Since the `IntentExecutor` is a permissionless shared singleton, any stranded ETH is immediately claimable by anyone calling `executeAndSweepNative()` with empty parameters

The caller who produced the ETH has no way to recover it, and has no indication the loss occurred as the transaction succeeds silently.

This is especially dangerous in the `CATValidator` combined flow: the `CATValidator`'s `_compareOutcomes()` only checks ERC20 outcome balances. If a swap routes through a path that unwraps WETH as a mid-step and ETH ends up in the `IntentExecutor`, the outcome check can still pass (the final ERC20 output was delivered) while the ETH value is silently lost to the next opportunistic sweeper.

Recommendation: Snapshot the native balance before execution and revert if it increased:

```
function executeAndSweep(
    Approval[] calldata approvals,
    Call3[] calldata calls,
    SweepTarget[] calldata sweepTargets
) external nonReentrant returns (Result[] memory results) {
+   uint256 nativeBefore = address(this).balance;
    _safeApproveAll(approvals);
    results = _executeCalls(calls);
    _sweepAll(sweepTargets);
+   if (address(this).balance > nativeBefore) revert UnexpectedNativeBalance();
}
```

This forces callers to use `executeAndSweepNative()` when ETH output is possible, making the ETH handling explicit rather than silently lossy. The snapshot approach avoids a DoS vector from force-sent ETH (via `selfdestruct`) since it only catches ETH that increased during execution.

LI.FI: Acknowledged and added documentation about this behavior in [lifinance/catapultar#29](#)

Researcher: Acknowledged.

6.2 Gas Optimization

6.2.1 Redundant approve calls on already-maxed allowances waste gas

Context: [IntentExecutor.sol#L123-L130](#), [IntentExecutor.sol#L15](#)

Description: Function `_safeApproveAll()` unconditionally calls `safeApproveWithRetry(token, spender, type(uint256).max)` on every execution, even when the allowance is already `type(uint256).max` from a previous transaction. The inline comment claims subsequent calls are a "no-op read (~3-4k gas saved)" but is inaccurate.

Function `safeApproveWithRetry()` always makes a full CALL to `token.approve(spender, amount)`. On a repeat call with the same value, the token contract still:

- Executes SLOAD on the allowance slot (~100 gas, warm)
- Executes SSTORE with the same value (~100 gas, EIP-2200 no-change)
- Emits an Approval event (~1,500 gas for LOG3 + topics)
- Returns and encodes result (~100 gas)

For a solver that routes through 3-4 token-spender pairs repeatedly, this wastes ~6,000-8,000 gas per transaction entirely avoidable.

Recommendation: Check the current allowance with a cheap `STATICCALL` before calling `approve`. Skip the `approve` entirely if already at max.

LI.FI: Fixed in [lifinance/catapultar#30](#)

Researcher: Verified fix.

6.3 Informational

6.3.1 Residual or accidentally sent tokens in `IntentExecutor` can be stolen by anyone

Context: [IntentExecutor.sol#L25](#)

Description: The `IntentExecutor` contract is stateless and fully permissionless both `executeAndSweep()` and `executeAndSweepNative()` have no access control. Combined with the open `receive()` external payable function, any tokens or native ETH residing in the contract can be drained by any external caller at any time.

Recommendation: Consider documenting the design invariant explicitly in inline documentation.

LI.FI: Fixed in [lifinance/catapultar#29](#)

Researcher: Verified fix.